

Tema 3: PL/SQL Básico

1. Bloques PL/SQL

Estructura: DECLARE (opcional), BEGIN (obligatorio), EXCEPTION (opcional).

```
1 DECLARE
2   v_nom VARCHAR2(50); v_sal NUMBER := 0;
3 BEGIN
4   SELECT nom,sal INTO v_nom,v_sal FROM emp WHERE id=101;
5   DBMS_OUTPUT.PUT_LINE('Emp: ' || v_nom);
6 EXCEPTION
7   WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No existe');
8   WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error: '||SQLERRM);
9 END;
10 /
```

Tipos: Anónimo (sin nombre), Nombrado (proc/func/trigger), Anidado.

```
1 DECLARE v_ext NUMBER := 10;
2 BEGIN
3   DECLARE v_int NUMBER := 20;
4   BEGIN
5     NULL; -- v_ext y v_int disponibles
6   END;
7   -- v_int no existe aquí
8 END;
9 /
```

2. Variables

Tipos: NUMBER, VARCHAR2(n), CHAR(n), DATE, BOOLEAN, TIMESTAMP.

```
1 DECLARE
2   v_nom VARCHAR2(100); v_edad NUMBER(3);
3   v_sal NUMBER(10,2) := 3000.50; v_activo BOOLEAN := TRUE;
4   v_fecha DATE := SYSDATE; c_iva CONSTANT NUMBER := 0.21;
5   BEGIN NULL; END;
6 /
```

%TYPE (tipo de columna):

```
1 v_nom emp.nom%TYPE; v_sal emp.sal%TYPE;
```

%ROWTYPE (fila completa):

```
1 DECLARE
2   v_emp emp%ROWTYPE;
3 BEGIN
4   SELECT * INTO v_emp FROM emp WHERE id=101;
5   DBMS_OUTPUT.PUT_LINE(v_emp.nom||': ' ||v_emp.sal);
6 END;
7 /
```

Asignación: v_nom := 'Juan'; v_i := v_i + 1;

3. Control

IF-THEN-ELSE:

```
1 IF v_sal>5000 THEN v_cat:='Alto';
2 ELSIF v_sal>3000 THEN v_cat:='Medio';
3 ELSE v_cat:='Bajo'; END IF;
```

CASE:

```
1 CASE v_dia
2   WHEN 1 THEN DBMS_OUTPUT.PUT_LINE('Lunes');
3   WHEN 2 THEN DBMS_OUTPUT.PUT_LINE('Martes');
4   ELSE DBMS_OUTPUT.PUT_LINE('Otro'); END CASE;
```

LOOP:

```
1 LOOP
2   v_i := v_i + 1;
3   EXIT WHEN v_i > 10;
4 END LOOP;
```

WHILE/FOR:

```
1 WHILE v_i <= 5 LOOP v_i:=v_i+1; END LOOP;
2 FOR i IN 1..10 LOOP DBMS_OUTPUT.PUT_LINE(i); END LOOP;
3 FOR i IN REVERSE 1..5 LOOP NULL; END LOOP;
```

4. Cursores

Implícito (1 fila):

```
1 SELECT nom,sal INTO v_nom,v_sal FROM emp WHERE id=101;
2 DBMS_OUTPUT.PUT_LINE('Filas: ' || SQL%ROWCOUNT);
```

Explícito (múltiples):

```
1 DECLARE
2   CURSOR c IS SELECT nom,sal FROM emp WHERE dept=10;
3   v_nom emp.nom%TYPE; v_sal emp.sal%TYPE;
4 BEGIN
5   OPEN c;
6   LOOP
7     FETCH c INTO v_nom,v_sal;
8     EXIT WHEN c%NOTFOUND;
9     DBMS_OUTPUT.PUT_LINE(v_nom||': ' ||v_sal);
10    END LOOP;
11   CLOSE c;
12 END;
13 /
```

FOR LOOP (automático):

```
1 FOR r IN (SELECT nom,sal FROM emp) LOOP
2   DBMS_OUTPUT.PUT_LINE(r.nom);
3 END LOOP;
```

Atributos: %FOUND, %NOTFOUND, %ROWCOUNT, %ISOPEN.

5. Registros

TYPE RECORD: Estructura de datos compuesta.

```
1 DECLARE
2   TYPE r_persona IS RECORD(
3     nombre VARCHAR2(25),
4     apellido VARCHAR2(25),
5     salario NUMBER(10));
6   v_emp r_persona;
7 BEGIN
8   v_emp.nombre := 'Ana';
9   v_emp.salario := 30000;
10 END;
11 /
```

%ROWTYPE para registros:

```
1 DECLARE
2   v_emp emp%ROWTYPE; -- Estructura de tabla
3   CURSOR c IS SELECT nom,sal FROM emp;
4   v_datos c%ROWTYPE; -- Estructura de cursor
5 BEGIN
6   SELECT * INTO v_emp FROM emp WHERE id=101;
7   DBMS_OUTPUT.PUT_LINE(v_emp.nom || ': ' || v_emp.sal);
8 END;
9 /
```

Cursores con registros:

```
1 DECLARE
2   CURSOR c IS SELECT nom,sal FROM emp;
3 BEGIN
4   FOR v_reg IN c LOOP
5     DBMS_OUTPUT.PUT_LINE(v_reg.nom || ': ' || v_reg.sal);
6   END LOOP;
7 END;
8 /
```

6. Excepciones

Predefinidas: NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, VALUE_ERROR, INVALID_NUMBER.

```
1 BEGIN
2   SELECT sal INTO v_sal FROM emp WHERE id=999;
3 EXCEPTION
4   WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No existe');
5   WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('Múltiples');
6   WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error: '||SQLERRM);
7 END;
8 /
```

Usuario:

```
1 DECLARE
2   edad_inv EXCEPTION;
3 BEGIN
4   IF v_edad<18 THEN RAISE edad_inv; END IF;
5 EXCEPTION
6   WHEN edad_inv THEN DBMS_OUTPUT.PUT_LINE('Edad >=18');
7 END;
8 /
```

7. Operaciones SQL

SELECT INTO:

```
1 SELECT nom,sal INTO v_nom,v_sal FROM emp WHERE id=101;
```

INSERT/UPDATE/DELETE:

```
1 INSERT INTO emp VALUES(999,'Ana',3500); COMMIT;
2 UPDATE emp SET sal=sal*1.1 WHERE dept=10;
3 DELETE FROM emp WHERE sal<1000;
4 DBMS_OUTPUT.PUT_LINE('Filas: ' || SQL%ROWCOUNT);
```

8. DBMS_OUTPUT

```
1 SET SERVEROUTPUT ON;
2 BEGIN
3   DBMS_OUTPUT.PUT_LINE('Mensaje');
4 END;
5 /
```

9. Procedimientos

CREATE PROCEDURE: Bloque con nombre almacenado en la BD.

```
1 CREATE OR REPLACE PROCEDURE actualizar_sal(  
2   p_id IN NUMBER,  
3   p_porc IN NUMBER  
4 ) IS  
5 BEGIN  
6   UPDATE emp SET sal=sal*(1+p_porc/100) WHERE id=p_id;  
7   COMMIT;  
8 END;  
9 /
```

Ejecución:

```
1 EXECUTE actualizar_sal(101, 10);  
2 -- 0 desde bloque:  
3 BEGIN actualizar_sal(101, 10); END;  
4 /
```

Parámetros: IN (entrada), OUT (salida), IN OUT (ambos).

10. Funciones

CREATE FUNCTION: Similar a procedimiento pero retorna valor.

```
1 CREATE OR REPLACE FUNCTION calc_iva(  
2   p_precio NUMBER  
3 ) RETURN NUMBER IS  
4 BEGIN  
5   RETURN p_precio * 1.21;  
6 END;  
7 /
```

Uso:

```
1 SELECT nom, precio, calc_iva(precio) FROM productos;  
2 -- 0 en bloque:  
3 v_total := calc_iva(100);
```

11. Ejemplos Prácticos

Calcular bonus:

```
1 DECLARE  
2 CURSOR c IS SELECT emp_id,sal FROM emp WHERE dept=10;  
3 BEGIN  
4   FOR r IN c LOOP  
5     UPDATE emp SET bonus=r.sal*0.1 WHERE emp_id=r.emp_id;  
6     END LOOP;  
7     COMMIT;  
8 END;  
9 /
```

Salario mínimo:

```
1 DECLARE  
2   c_min CONSTANT NUMBER := 1500;  
3 BEGIN  
4   UPDATE emp SET sal=c_min WHERE sal<c_min;  
5   DBMS_OUTPUT.PUT_LINE('Actualizados: ' || SQL%ROWCOUNT);  
6   COMMIT;  
7 END;  
8 /
```